```
  {
   Auth::extend('jwt', function (Application $app, string $name,
   array $config) {
    // Return an instance of Illuminate\Contracts\Auth\Guard
   return new
   JwtGuard(Auth::createUserProvider($config['provider']));
   });
  }
}
```

As seen in the preceding example, the callback supplied to the **`extend`** method should return an **`Illuminate\Contracts\Auth\Guard`** implementation. This interface provides a few methods that must be implemented to define a custom guard. Once you've defined your own guard, you can use it in the **`guards`** configuration of your **`auth.php`** configuration file:

```
'guards' => [
 'api' => [
  'driver' => 'jwt',
  'provider' => 'users',
 ],
],
```

Closure Request Guards

Using the **`Auth::viaRequest`** function is the simplest approach to construct a bespoke HTTP request-based authentication system. Using a single closure, you can easily create your authentication procedure.

To begin, invoke the **`Auth::viaRequest`** function from your **`AuthServiceProvider`**'s startup method. The first argument to the **`viaRequest`** method is the name of an authentication driver. Any string that describes your custom guard can be used as this name. The method's second parameter should be a closure that receives the incoming HTTP request and returns a user instance or **`null`** if authentication fails:

```
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
/**
```